

eRakun — Dokumentacija projekta

Kolegij: Razvoj IT rješenja

Alan Bubalo

lipanj 2026.

Sažetak

eRakun je REST API aplikacija u Laravelu koja oponaša rad informacijskog posrednika (access pointa) u hrvatskoj Fiskalizaciji 2.0. Pokriva cijeli 4-corner tijekom razmjene eRačuna: prijem računa od ERP-a, generiranje i parsiranje UBL 2.1 / HR-CIUS dokumenata, pravo digitalno potpisivanje i verifikaciju na dva sloja (XAdES-B na dokumentu, WS-Security na transportu) s vlastitim testnim PKI-jem, fiskalizaciju prema mock CIS-u sa sravnjivanjem prijava te AS4 razmjenu između dva posrednika s otkrivanjem primatelja preko AMS/MPS-a. Projekt je izrađen kao edukacijska vježba: vjerno oponaša arhitekturu, formate i tokove poruka certificiranog posrednika, dok su vanjski sustavi (Porezna/CIS, drugi posrednik, nacionalni adresar) i korijeni povjerenja simulirani. Dokument opisuje domenski kontekst, arhitekturu, use case-ove, model podataka, API i sigurnosni (PKI) sloj, sve izvedeno iz stvarnog koda.

Sadržaj

Sažetak	1
1 Uvod	3
1.1 Što je eRakun	3
1.2 Kontekst — Fiskalizacija 2.0	3
1.3 Opseg i ograničenja	3
1.4 Tehnološki stog	3
2 Motivacija	4
2.1 Ciljano tržište	4
2.2 Dosadašnji proces i postojeća rješenja	4
2.3 Predispozicije za uvođenje	4
2.4 Tko ima koristi	5
2.5 SWOT analiza	5
3 Domenski kontekst	7

3.1	Pojmovnik	7
3.2	4-corner model	7
4	Arhitektura sustava	8
4.1	Dvije aplikacije	8
4.2	Slojevi i obrasci	8
4.3	Tijek izlaznog računa (high-level)	9
4.4	Stanja i prijelazi	9
4.5	Konfiguracija i zamjenjivost (DI)	10
4.6	Sigurnost: PKI i potpisi	10
5	Use case-ovi	11
5.1	Krovni Use Case dijagram	12
5.2	A. Registar i postavljanje	12
5.3	B. Izlazni račun	15
5.4	C. Ulazni račun	18
6	Model podataka	20
6.1	Klasni dijagram domene	20
6.2	Tablice i ključne invarijante	21
7	Korisničke upute	22
7.1	Sučelje: Bruno API kolekcija	22
7.2	Priprema okoline	23
7.3	Tijek rada (pregled)	23
7.4	Korak 1 — Registracija stranaka	23
7.5	Korak 2 — Potpisni certifikat dobavljača	24
7.6	Korak 3 — Kreiranje izlaznog računa	24
7.7	Korak 4 — Generiranje i potpisivanje UBL-a	25
7.8	Korak 5 — Fiskalizacija i AS4 dostava	25
7.9	Korak 6 — Zaprimanje i mrežne usluge	25
7.10	Čitanje statusa i česte greške	26
8	Zaključak i ograničenja	26
8.1	Što projekt postiže	26
8.2	Svjesna ograničenja	27
9	Literatura i izvori	27

1 Uvod

1.1 Što je eRakun

eRakun je aplikacija (REST API) napisana u Laravelu (PHP) koja oponaša rad informacijskog posrednika u sustavu hrvatske Fiskalizacije 2.0. Posrednik je certificirani poslužitelj (*access point*) koji stoji između poslovnih subjekata i Porezne uprave te omogućuje elektroničku razmjenu računa (eRačuna) i njihovo prijavljivanje državi.

Projekt je izrađen kao edukacijska vježba: cilj je razumjeti i implementirati stvarne tokove poruka, formate i protokole sustava eRačuna, a ne pustiti sustav u produkciju. Zato su sve vanjske strane oponašane (vidi 1.3).

1.2 Kontekst — Fiskalizacija 2.0

Od 1. siječnja 2026. hrvatski zakon o Fiskalizaciji 2.0 obvezuje sve obveznike PDV-a na elektroničku razmjenu računa u B2B i B2G prometu. Umjesto izravne razmjene PDF-ova ili papira, računi se izmjenjuju kao strukturirani XML dokumenti kroz mrežu certificiranih posrednika, a paralelno se prijavljuju (fiskaliziraju) Poreznoj upravi. eRakun implementira upravo ulogu jednog takvog posrednika.

1.3 Opseg i ograničenja

Projekt implementira funkcionalnu jezgru posrednika:

- prijem računa od ERP sustava (REST API, „unutarnji rub”)
- generiranje i parsiranje UBL 2.1 dokumenata s hrvatskim ekstenzijama
- digitalno potpisivanje i verifikaciju potpisa (pravi kriptografski potpis, vlastiti testni PKI)
- fiskalizaciju prema mock CIS-u (Porezna)
- razmjenu računa s drugim posrednikom preko AS4 protokola
- otkrivanje primatelja (discovery) preko mock nacionalnog adresara (AMS/MPS)

Sve vanjske strane (Porezna/CIS, drugi posrednik, nacionalni adresar) i korijeni povjerenja oponašani su radi cjelovite demonstracije, dok domenski kod ostaje produkcijski. Potpuni popis svjesnih granica opsega — mock CIS, testni PKI, podskup eReportinga, izostanak autentikacije API-ja i drugo — naveden je na jednom mjestu u poglavlju 8 (Svjesna ograničenja).

1.4 Tehnološki stog

Sloj	Tehnologija
Jezik / framework	PHP 8.2+, Laravel 12
Baza podataka	relacijska (Eloquent ORM, migracije); SQLite u razvoju
Format računa	UBL 2.1 XML (DOM manipulacija)
Kriptografija	OpenSSL, <code>robrichards/xmlseclibs</code> (XML-DSig / XAdES)
Transport (posrednik↔posrednik)	AS4 (ebMS 3.0) preko HTTP-a
Testiranje	Pest; statička analiza Larastan/PHPStan; Rector; Pint

2 Motivacija

2.1 Ciljano tržište

Zakon o fiskalizaciji uvodi Fiskalizaciju 2.0 kao obvezu od 1. siječnja 2026. Od tog datuma svi obveznici PDV-a u Hrvatskoj moraju izdavati i primati račune u B2B (poslovni subjekt → poslovni subjekt) i B2G (prema javnom sektoru) prometu isključivo kao strukturirane elektroničke račune (eRačune), uz istodobnu prijavu (fiskalizaciju) podataka Poreznoj upravi. Razmjena više nije izravna, nego se odvija kroz mrežu certificiranih informacijskih posrednika (access pointova).

Ciljano tržište aplikacije ovog tipa stoga su informacijski posrednici i subjekti koji to žele postati: računovodstveni servisi, dobavljači ERP i fakturnih sustava te veći poslovni subjekti koji žele opsluživati sebe i svoje klijente. Krajnji korisnici (obveznici PDV-a) neizravno su tržište jer se na posrednika *priključuju*, izravno ili kroz svoj ERP.

Ova motivacija opisuje stvarno tržište i vrijednost koju rješenje ovog tipa donosi; gdje je relevantno, istaknuto je što u eRakunu jest, a što je svjesno oponašano (8).

2.2 Dosadašnji proces i postojeća rješenja

Dosadašnji proces (prije Fiskalizacije 2.0). U B2B prometu račun se najčešće slao kao nestrukturirani dokument, papirom poštom ili kao PDF privitak u e-pošti. Primatelj bi podatke ručno prepisivao u svoj sustav (ili u boljem slučaju kroz OCR), uz česte pogreške, gubitak dokumenata, sporu obradu i otežanu reviziju. Prijava prometa državi odvijala se naknadno, kroz periodička izvješća (PDV obrasci), bez razmjene pojedinačnih računa u stvarnom vremenu. B2G promet već je dijelom bio digitaliziran (obvezni eRačun prema javnim naručiteljima preko servisa kao što je FINA-in), ali B2B je ostao pretežno ručan.

Postojeća i konkurentska rješenja. Tržište posrednika nije prazno. Porezna uprava vodi službeni popis registriranih informacijskih posrednika. Glavne skupine igrača su:

- **FINA** (Financijska agencija) državna je agencija koja izdaje kvalificirane certifikate i sama nudi servis eRačuna te ulogu posrednika, pa je de facto referentna točka.
- **Komercijalni posrednici** (npr. Moj-eRačun i sl.) etablirane su platforme za razmjenu eRačuna s velikom postojećom bazom korisnika.
- **Dobavljači ERP / računovodstvenih sustava** certificiraju se kao posrednici kako bi svojim klijentima ponudili razmjenu eRačuna „ugrađenu” u postojeći softver.

Zajedničko im je da se svi oslanjaju na iste standarde (UBL 2.1 / HR-CIUS, EN 16931, PEP-POL AS4, fiskalne poruke prema CIS-u). Diferencijacija je u integracijama, cijeni, korisničkom iskustvu i dodatnim uslugama (arhiviranje, analitika, povezivanje s plaćanjem), a ne u samom protokolu, što interoperabilnost čini obveznom, a ulaz tehnički zahtjevnim ali jasno specificiranim.

2.3 Predispozicije za uvođenje

Da bi posrednik ovakvog tipa zaživio u produkciji, moraju biti zadovoljene vanjske pretpostavke, odnosno sustavi i vjerodajnice koje pruža okolina:

Predispozicija	Opis
Registracija posrednika	Upis u službeni registar Porezne uprave i u OpenPEPPOL mrežu (access point).
Kvalificirani certifikati	Certifikati stranaka (vezani uz OIB) i certifikat samog posrednika (transportni), izdani od ovlaštenog tijela (npr. FINA).
CIS dostupan kao web servis	Centralni informacijski sustav Porezne mora primati fiskalne poruke i vraćati potvrde s rezultatom uparivanja.
Nacionalni adresar (AMS/MPS)	Imenik za otkrivanje koji posrednik opslužuje koji OIB, da pošiljatelj zna kamo dostaviti račun.
Spremnost krajnjih korisnika	Obveznici (ili njihovi ERP-ovi) moraju moći proizvesti/zaprimiti UBL i imati valjan OIB.

U eRakunu su sve vanjske predispozicije oponašane radi cjelovite demonstracije: CIS i AMS zamjenjuje companion aplikacija **erakun-porezna**, a kvalificirane certifikate vlastiti testni PKI (zamjenjivost mock ↔ produkcija: 4.5).

2.4 Tko ima koristi

Uvođenje posrednika i eRačuna donosi korist širem krugu od samih krajnjih korisnika:

- **Poslovni subjekti (obveznici PDV-a)** dobivaju bržu i jeftiniju obradu, manje ručnog unosa i pogrešaka, automatsko uparivanje s narudžbama te lakše ispunjavanje zakonske obveze.
- **Računovodstveni servisi** dobivaju strukturirane podatke koji ulaze izravno u knjigovodstvo, bez prepisivanja s PDF-a.
- **Porezna uprava (država)** dobiva uvid u promet u stvarnom vremenu, automatsko uparivanje prijava dobavljača i kupca te smanjenje porezne utaje i PDV jaza.
- **Sami posrednici** dobivaju novu tržišnu ulogu i izvor prihoda (naknada po računu ili pretplata) na zakonom potaknutom tržištu.
- **Šire gospodarstvo** dobiva kraće rokove naplate, manje papira, bolju revizijsku sljedivost i interoperabilnost s EU-om (PEPPOL).

2.5 SWOT analiza

SWOT analiza promatra eRakun kao kandidat-rješenje informacijskog posrednika na opisanom tržištu, pri čemu su snage i slabosti unutarnje (svojstva samog rješenja), a prilike i prijetnje vanjske (tržište i okolina).

Snage (Strengths) (unutarnje, pozitivno)

Vjernost standardima	Implementira prave standarde (UBL 2.1 / HR-CIUS, EN 16931, AS4, XAdES-B, WS-Security), pa je tehnička prepreka za prelazak na produkciju niska.
Slojevita arhitektura	Domena je odvojena od HTTP-a i transporta, a vanjski servisi su iza sučelja (4.2), pa je mock zamjenjiv pravim sustavom bez diranja domene.
Prava kriptografija	RSA-2048/SHA-256 potpisi i <i>stvarna</i> verifikacija s provjerom lanca povjerenja, na oba sloja (dokument + transport).
Otpornost	Idempotentne radnje, zaštita od ponavljanja (replay), odvojena (decoupled) dostava i fiskalizacija s ručnim ponavljanjem.

Slabosti (Weaknesses) (unutarnje, negativno)

Edukacijski opseg	CIS/AMS i PKI su oponašani, pa rješenje nije certificirano ni povezano s pravim FINA/OpenPEPPOL korijenima (puni popis granica opsega: 8).
Sigurnost API-ja	Nema autentikacije/autorizacije ERP klijenata, rate-limitinga ni multi-tenancyja.
Bez GUI-a	Sustav je REST API i nema gotovog grafičkog sučelja za netehničke korisnike.

Prilike (Opportunities) (vanjske, pozitivno)

Zakonska obveza	Cijelo tržište obveznika PDV-a prisiljeno je na eRačun, što jamči potražnju.
Ugradnja u ERP	Mogućnost ponude kao „white-label” motor unutar postojećih ERP/računovodstvenih sustava.
Prekogranično (EU)	PEPPOL temelj omogućuje proširenje na prekograničnu razmjenu unutar EU-a.
Dodatne usluge	Arhiviranje, analitika te povezivanje s plaćanjem i statusima izvori su dodatne vrijednosti.

Prijetnje (Threats) (vanjske, negativno)

Etabilirana konkurencija	FINA, Moj-eRačun i veliki ERP dobavljači imaju prednost prvog ulaska i postojeću bazu korisnika.
Troškovi sukladnosti	Certifikacija, sigurnosni zahtjevi i kontinuirano održavanje sukladnosti su skupi.
Promjene specifikacija	HR-CIUS, KPD i fiskalni format se mijenjaju, što zahtijeva stalno održavanje.
Sigurnosni rizik	Rukovanje privatnim ključevima i certifikatima stranaka nosi visok rizik kompromitacije.

Zaključak SWOT-a: na zakonom zajamčenom tržištu (prilika) glavna vrijednost rješenja je vjernost standardima i čista arhitektura (snage), dok su ključni rizici etabrirana konkurencija i trošak sukladnosti. Slabosti su namjerne granice edukacijskog opsega, jasno označene da se ne prenose u produkciju (poglavlje 8).

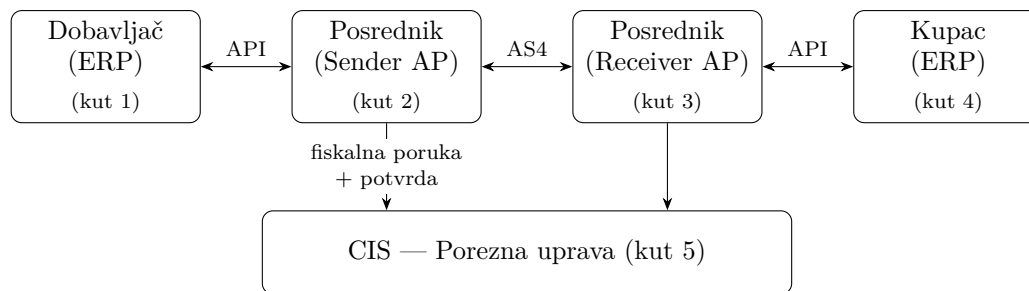
3 Domenski kontekst

3.1 Pojmovnik

Pojam	Značenje
Informacijski posrednik	Certificirani access point koji šalje/prima eRačune i fiskalizira ih. Uloga koju glumi eRakun.
eRačun	Strukturirani elektronički račun (UBL 2.1 XML), pravno ekvivalentan papirnatom.
Fiskalizacija	Prijava računa CIS-u; obje strane prijavljuju isti račun, a CIS provodi uparivanje (matching) i vraća potvrdu.
CIS	Centralni informacijski sustav Porezne uprave. U projektu: mock erakun-porezna .
Party (strana)	Poslovni subjekt — dobavljač (<i>supplier</i>) ili kupac (<i>buyer</i>). Identificiran OIB-om.
OIB	Osobni identifikacijski broj — 11 znamenki, kontrolna znamenka po ISO 7064 (MOD 11,10).
UBL 2.1	Universal Business Language — XML standard za poslovne dokumente.
HR-FISK 2.0 / HR-CIUS	Hrvatska prilagodba (<i>Core Invoice Usage Specification</i>) standarda EN 16931.
EN 16931	Europska norma za elektroničko fakturiranje (semantički model računa).
KPD 2025	Klasifikacija proizvoda po djelatnostima — 6-znamenkasti kôd na svakoj stavci računa.
AS4	Protokol za sigurnu B2B razmjenu poruka (ebMS 3.0); transport između posrednika.
AMS / MPS	Adresar / Metapodatkovni servis — imenik za otkrivanje endpointa primatelja po OIB-u.
CA	<i>Certificate Authority</i> — certifikacijsko tijelo koje izdaje i potpisuje certifikate; vrh lanca povjerenja. Korijska (root) CA je samopotpisana. U projektu: testne FINA-like i PEPPOL-like CA (u produkciji prave FINA i OpenPEPPOL).
CN	<i>Common Name</i> — polje u subjektu X.509 certifikata koje identificira vlasnika. U projektu se OIB stranke upisuje kao CN (kao kod prave FINA-e).
Standardi po slojevima	Format: UBL 2.1 / HR-CIUS (semantika EN 16931-1:2017); transport: AS4 (ebMS 3.0); potpis: XAdES-B (dokument) + WS-Security (transport); klasifikacija: KPD 2025; otkrivanje: AMS + MPS.

3.2 4-corner model

Razmjena eRačuna odvija se kroz četiri ugla (peti je Porezna). Pošiljalac nikad ne šalje izravno primatelju — sve ide preko posrednika:



- **Kut 1 — Dobavljač (ERP):** izdaje račun, šalje ga svom posredniku preko REST API-ja.
- **Kut 2 — Posrednik pošiljatelj (Sender AP):** validira, potpisuje i pretvara račun u UBL; fiskalizira ga prema CIS-u; otkriva primateljev posrednik i šalje ga preko AS4.
- **Kut 3 — Posrednik primatelj (Receiver AP):** prima AS4 poruku, raspakira UBL, (po potrebi) fiskalizira sa svoje strane i dostavlja kupcu.
- **Kut 4 — Kupac (ERP):** preuzima zaprimljeni račun preko API-ja.
- **Kut 5 — CIS / Porezna:** prima fiskalne poruke, provodi uparivanje i vraća potvrdu s identifikatorom poruke.

eRakun je jedna aplikacija koja pokriva i kut 2 i kut 3 — ista instanca može biti pošiljatelj za jedne i primatelj za druge račune. Za demonstraciju punog round-tripa pokreću se dvije instance eRakuna (sender + receiver).

4 Arhitektura sustava

4.1 Dvije aplikacije

Aplikacija	Uloga
erakun (ovaj repozitorij)	Informacijski posrednik. Pokriva kut 2 i kut 3. Sadrži i MPS (vlastiti metapodatkovni servis).
erakun-porezna (companion)	Oponaša državne servise: CIS (prima fiskalne poruke, vraća potvrdu s identifikatorom poruke <code>service_message_id</code>) i AMS (nacionalni adresar za discovery).

Komunikacija eRakun → Porezna ide preko HTTP-a; svaki vanjski servis skriven je iza sučelja, pa je mock zamjenjiv pravim sustavom bez diranja domene (mehanizam u 4.5).

4.2 Slojevi i obrasci

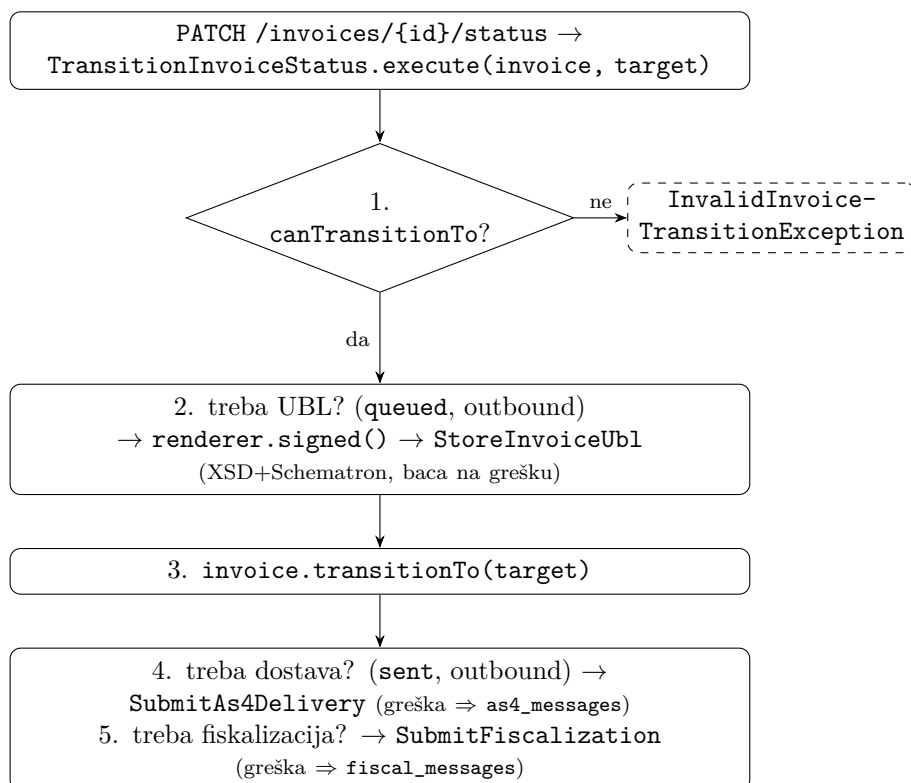
Projekt slijedi Laravel konvencije uz svjesnu odluku da kontroleri ostanu tanki: izlažu samo REST resource rute, a svu poslovnu logiku nose Action klase (`app/Actions/`). Radnje koje nisu čisti CRUD (fiskalizacija, AS4 dostava) dobivaju vlastiti kontroler umjesto proizvoljnih metoda na jednom kontroleru. Time je domenska logika izolirana od HTTP sloja i lako testabilna, a vidljiva je i u use case-ovima (svaki navodi pripadnu Action klasu).

Gruba podjela odgovornosti po direktorijima:

Direktorij	Odgovornost
app/Http/Controllers	Tanki REST kontroleri
app/Http/Requests	Validacija ulaza (form requests)
app/Http/Resources	Oblikovanje JSON odgovora
app/Actions	Poslovne radnje (domenska jezgra)
app/Models	Eloquent modeli + invarijante stanja
app/Enums	Stanja, smjerovi, tipovi (vidi 4.4)
app/Validation	UBL XSD/Schematron validacija (<code>UblValidator</code>)
app/Pki	Generiranje testnog PKI-ja, potpisne vjerodajnice, trust store
app/As4	AS4 transport (envelope build/sign, dostava, prijem, peer discovery)
app/Fiscalization	Klijent prema CIS-u, DTO-ovi odgovora, sravnjivanje
app/Rules	Domenska validacijska pravila (npr. <code>0ib</code>)

4.3 Tijek izlaznog računa (high-level)

Središnja orkestracija je `TransitionInvoiceStatus` — jedini siguran put za mijenjanje statusa računa. On prvo provjeri je li prijelaz dopušten, zatim (po potrebi) renderira i potpiše UBL prije promjene statusa (tako neispravan dokument ostavlja račun u zatečenom stanju), i tek onda okida side-effecte:



Ključna projektna odluka: dostava i fiskalizacija su odvojene (decoupled) — ako ne uspiju, prijelaz statusa se ne poništava, nego se greška trajno zapiše na pripadni red i izloži kroz API. Ručni ponovni pokušaj razrađen je u UC-B6/UC-B7.

4.4 Stanja i prijelazi

Domena je modelirana kroz enume — svaki entitet ima stanje, neki i smjer:

Enum	Vrijednosti	Uloga
InvoiceStatus	draft → queued → sent → received → delivered / rejected	Životni ciklus računa
InvoiceDirection	outbound, inbound	Šaljemo li ga ili primamo
FiscalMessageState	requested → accepted / error	Ishod fiskalizacije
FiscalMessageType	fis (+ predviđeni fis_payment, fis_reject, fis_ir)	Vrsta fiskalne poruke
As4MessageState	queued → sent → acknowledged → received → delivered / error	Stanje AS4 prijenosa
As4MessageDirection	outbound, inbound	Smjer AS4 poruke
MatchStatus	pending → matched / mismatch	Sravnjivanje pošiljatelj/primatelj fiskalizacije
CertificateStatus	active, superseded, revoked	Stanje potpisnog certifikata stranke
VatCategory	S (standard), Z (nulta stopa), E (oslobođeno)	PDV kategorija stavke

Dopušteni prijelazi definirani su na enumu `InvoiceStatus` (`allowedTransitions` / `canTransitionTo`), a `Invoice::transitionTo` je čuvani ulaz koji delegira na enum; detaljna matrica prijelaza dokumentirana je u poglavlju 5.3.

4.5 Konfiguracija i zamjenjivost (DI)

Sve vanjske ovisnosti registrirane su kao singletoni u `AppServiceProvider` i sakrivene iza sučelja, pa je mock zamjenjiv pravim servisom samo promjenom konfiguracije:

- `FiscalizationService` → `HttpFiscalizationService` (URL/timeout iz `services.fiscalization`)
- `As4DeliveryService` → `HttpAs4DeliveryService` (envelope builder + signer + peer resolver)
- `PeerEndpointResolver` → `AmsMpsPeerEndpointResolver` s fallbackom na `ConfigPeerEndpointResolver` (statička mapa `OIB=URL` iz konfiguracije) — discovery prvo pokušava AMS, pa padne na lokalnu mapu
- PKI: `TestPkiGenerator`, `AccessPointCredential`, `TrustStore` (vjeruje dvama testnim korijenima — „FINA-like” za stranke i „PEPPOL-like” za access pointe)

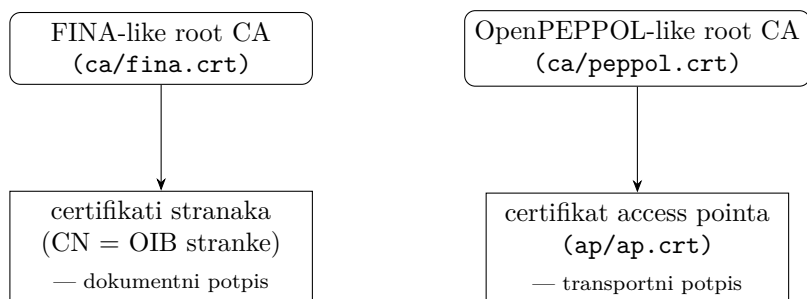
4.6 Sigurnost: PKI i potpisi

Potpisivanje i verifikacija su prava kriptografija (RSA-2048, SHA-256; OpenSSL + `robrichards/xmlseclibs`); jedino su korijeni povjerenja samo-generirani (testni PKI) umjesto pravih FINA/OpenPEPPOL korijena. Sustav vjerno odražava PEPPOL podjelu na dokumentni i transportni sloj, s dva odvojena certifikata — ulazni dokument nosi oba potpisa, koja se provjeravaju pri prijemu i ingestiji (usp. UC-C2):

Sloj	Što potpisuje	Čime	Standard	Kod
Dokumentni	UBL račun (enveloped)	certifikat stranke (FINA-like)	XAdES-B	InvoiceSigner
Transportni	AS4 envelope (payload)	certifikat access pointa (PEPPOL-like)	WS-Security	As4EnvelopeSigner

XAdES-B potpis je enveloped (reference na dokument i na `SignedProperties` sa `SigningTime`), računa se nad in-memory DOM-om i pohranjuje verbatim — naknadno „uljepšavanje” ubacilo bi bjeline koje digesti nisu vidjeli i tiho slomilo verifikaciju.

Testni PKI. `php artisan pki:generate` stvara dvije samopotpisane korijenske CA: certifikati stranaka izdaju se s FINA-like CA (CN = OIB, kao prava FINA), a certifikat access pointa s PEPPOL-like CA. Privatni ključevi pišu se na gitignoran PKI disk i nikad ne napuštaju poslužitelj ni API; `TrustStore` vjeruje upravo tim dvama korijenima.



Verifikacija (`SignatureVerifier`) je stvarna i radi tri provjere: **autentičnost** (`SignatureValue` provjeren javnim ključem iz ugrađenog certifikata), **integritet** (ponovni digest svake reference iz živog dokumenta uz `hash_equals` — svaka izmjena nakon potpisa daje `digestMismatch`) i **povjerenje** (lanac do povjerljivog CA + rok valjanosti), na obje strane razmjene. Pri potpisivanju se koristi jedini aktivni certifikat stranke; bez njega se baca iznimka — dokument nikad ne izlazi nepotpisan.

5 Use case-ovi

Ovo je središnje poglavlje. Use case-ovi su grupirani u tri cjeline:

- **A. Registar i postavljanje** — onboarding stranaka, potpisni certifikati, otkrivanje primatelja
- **B. Izlazni račun** — kreiranje, životni ciklus, UBL, fiskalizacija, AS4 dostava
- **C. Ulazni račun** — prijem iz ERP-a, prijem preko AS4, savnjivanje

Svaki use case opisan je predloškom: akteri, preduvjeti, glavni tok, posljedice/side-effecti te alternativni tokovi i greške. Tokovi su izvedeni iz stvarnog koda (Action klase, kontroleri, modeli), ne iz razvojnih bilješki.

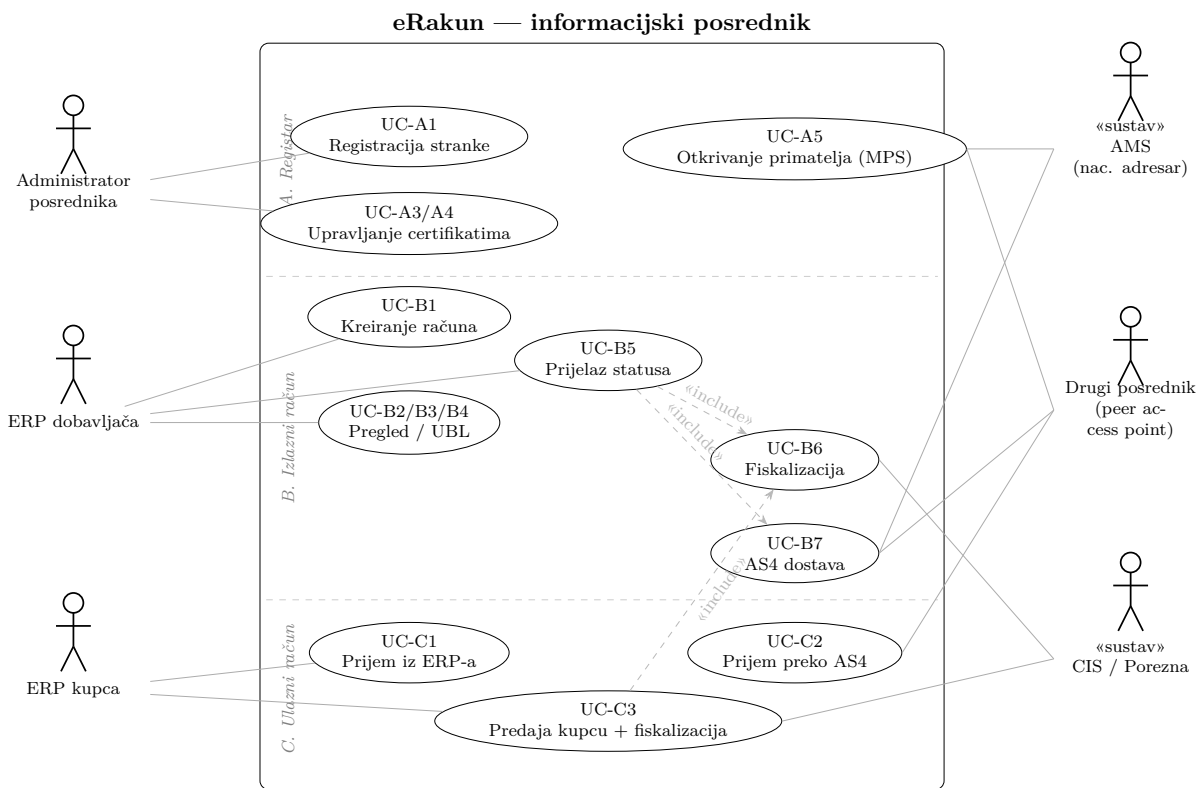
Akteri u sustavu:

- **ERP dobavljača / kupca** — vanjski sustav koji preko REST API-ja komunicira s posrednikom.
- **Administrator posrednika** — onboarda stranke i učitava certifikate.
- **Drugi posrednik (peer access point)** — šalje/prima AS4 poruke.

- **CIS (Porezna)** — prima fiskalne poruke, provodi uparivanje, vraća potvrdu s identifikatorom poruke. (mock: `erakun-porezna`)
- **AMS (nacionalni adresar)** — imenik za otkrivanje endpointa. (mock: `erakun-porezna`)

5.1 Krovni Use Case dijagram

Dijagram prikazuje cijeli sustav: lijevo su ljudski/ERP akteri, desno vanjski sustavi («sustav») s kojima posrednik komunicira, a u sredini su use case-ovi grupirani u tri cjeline (A — registar, B — izlazni račun, C — ulazni račun). Radi preglednosti krovni dijagram spaja srodne čitateljske radnje u jedan use case (npr. *Pregled / dohvat računa* pokriva UC-B2/B3/B4); pojedinačni use case-ovi razrađeni su u nastavku poglavlja (UC-A1 ... UC-C3).



Komunikacija s vanjskim sustavima (desni akteri) vidljiva je izravno na dijagramu: CIS prima fiskalne poruke (UC-B6, UC-C3), drugi posrednik razmjenjuje AS4 poruke (UC-B7 izlaz, UC-C2 ulaz) i pita naš MPS (UC-A5), a AMS se koristi pri najavi stranke (UC-A1) i otkrivanju primatelja (UC-B7). Prijelaz statusa (UC-B5) je orkestracijski use case koji «include»-a fiskalizaciju i dostavu; predaja kupcu (UC-C3) «include»-a fiskalizaciju sa strane primatelja.

5.2 A. Registar i postavljanje

Prije nego što ijedan račun može proteći kroz sustav, posrednik mora poznavati stranke (dobavljače i kupce) i imati njihove potpisne certifikate. Ova cjelina pokriva i otkrivanje (discovery) — kako jedan posrednik pronalazi gdje dostaviti račun za zadani OIB.

5.2.1 UC-A1 — Registracija stranke (onboarding)

Akteri	Administrator posrednika / ERP
Cilj	Upisati poslovni subjekt (dobavljača ili kupca) u registar posrednika.
Endpoint	POST /parties
Kod	PartyController@store → RegisterParticipant

Preduvjeti: nijedan (osim valjanog tijela zahtjeva).

Glavni tok:

1. ERP/administrator šalje podatke o stranci: `oib`, `name`, `address_line`, `city`, `postcode` (5 znakova), opcionalno `is_vat_registered`, `country_code` (2 znaka), `iban`.
2. `StorePartyRequest` validira ulaz: OIB se provjerava pravilom 0ib (11 znamenki + kontrolna znamenka ISO 7064, MOD 11,10) i mora biti jedinstven (`unique:parties,oib`).
3. Stranka se kreira u tablici `parties`.
4. **Najava u AMS-u:** `RegisterParticipant` objavljuje stranku u nacionalnom adresaru kao mapiranje OIB → MPS base URL ovog posrednika, kako bi drugi posrednici znali da mi opslužujemo taj OIB.
5. Vraća se 201 Created s `PartyResource` prikazom stranke.

Posljedice: novi red u `parties`; (po uspjehu) zapis u AMS-u da ovaj posrednik opslužuje dati OIB.

Alternativni tokovi i greške:

- Nevaljan/duplikat OIB ili nedostajuća polja → 422 Unprocessable Entity (Laravel validacija).
- **AMS najava je best-effort:** ako MPS base URL nije konfiguriran (`RegisterParticipant` vrati `false`) ili je AMS nedostupan, onboarding svejedno uspijeva — registracija stranke se ne poništava zbog neuspjele najave.

5.2.2 UC-A2 — Dohvat stranke

Akteri	ERP / administrator
Cilj	Dohvatiti podatke o registriranoj stranci.
Endpoint	GET /parties/{party}
Kod	PartyController@show

Glavni tok: dohvat stranke po identifikatoru i povrat `PartyResource` prikaza (`id`, `oib`, `name`, `adresa`, `is_vat_registered`, `country_code`, `iban`). Nepostojeća stranka → 404.

5.2.3 UC-A3 — Učitavanje potpisnog certifikata stranke

Akteri	Administrator posrednika
Cilj	Registrirati certifikat kojim se potpisuju dokumenti u ime stranke (dokumentni sloj).
Endpoint	POST /parties/{party}/certificates
Kod	CertificateController@store → StoreCertificate

Preduvjeti: stranka postoji.

Glavni tok:

1. Administrator šalje certifikat na jedan od dva načina (`StoreCertificateRequest`):
 - datoteka `certificate` — PKCS#12 (`.p12/.pfx`, do 50 MB), ili
 - polje `certificate_pem` — PEM string koji nosi certifikat (po želji i privatni ključ),
 - uz opcionalni `password`.
2. `StoreCertificate.normalize()` prepoznaje format (binarni PKCS#12 vs. PEM s `---BEGIN---` omotom) i normalizira na PEM, izdvajajući privatni ključ i certifikat (OpenSSL).
3. `CertificateMetadata::fromPem()` iz certifikata izvlači metapodatke: serijski broj, subject, issuer, rok valjanosti (`valid_from/valid_to`), fingerprint.
4. **Pohrana ključa:** privatni ključ i certifikat zapisuju se na gitignoran PKI disk pod `parties/{oib}/{serial}.key` i `.crt`.
5. **Invarijanta „samo jedan aktivan“:** u jednoj transakciji prethodni Active certifikat stranke prelazi u Superseded, a novi se kreira kao Active.
6. Vraća se 201 Created s `CertificateResource`.

Posljedice: novi red u `certificates` (status active); eventualni prethodni aktivni postaje superseded; privatni ključ pohranjen na PKI disku (nikad se ne vraća kroz API).

Alternativni tokovi i greške:

- Pogrešna lozinka ili neispravan PKCS#12 → 422 („Unable to read PKCS#12...”).
- PEM bez valjanog X.509 certifikata ili bez čitljivog privatnog ključa → 422.

5.2.4 UC-A4 — Pregled, dohvat i opoziv certifikata

Akteri	Administrator posrednika
Cilj	Vidjeti certifikate stranke, dohvatiti pojedini, opozvati certifikat.
Endpointi	GET <code>/parties/{party}/certificates</code> , GET <code>.../{certificate}</code> , DELETE <code>.../{certificate}</code>
Kod	<code>CertificateController@index</code> / <code>show</code> / <code>destroy</code>

Glavni tokovi:

- **Pregled** (`index`): lista certifikata stranke, najnoviji prvi.
- **Dohvat** (`show`): pojedini certifikat (scoped binding veže `{certificate}` uz `{party}`).
- **Opoziv** (`destroy`): certifikatu se status postavlja na `revoked` i vraća se 204 No Content. Opoziv je „meki” — red se ne briše, čime ostaje revizijski trag, a potpisani dokumenti nastali dok je bio aktivan i dalje su provjerivi.

5.2.5 UC-A5 — Otkrivanje primatelja (MPS lookup)

Akteri	Drugi posrednik (peer)
Cilj	Saznati kako (na kojem AS4 endpointu) dostaviti račun za zadani OIB.
Endpoint	GET <code>/mps/participants/{oib}</code>
Kod	<code>MpsParticipantController@show</code>

Koncept: MPS (*Metapodatkovni servis*) ovog posrednika objavljuje koje stranke opslužujemo. Nema vlastitu tablicu — odgovor je izveden iz `parties` tablice i konfiguracije.

Glavni tok:

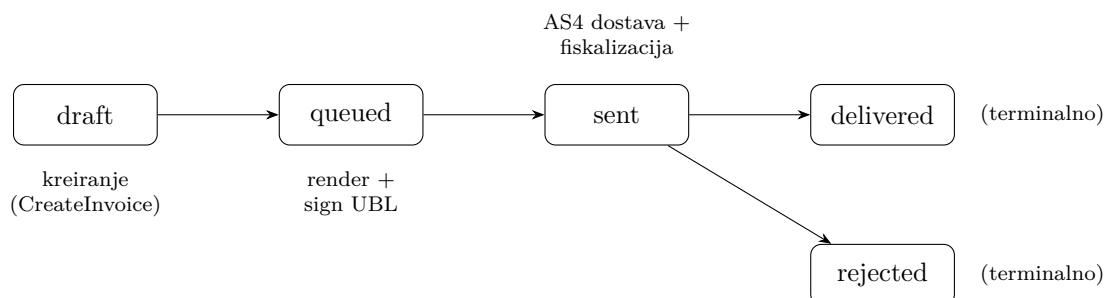
1. Peer (ili `AmsMpsPeerEndpointResolver` drugog posrednika) traži OIB.
2. Ako stranku s tim OIB-om opslužujemo (`parties` sadrži OIB) → vraća se JSON `{ oib, as4_endpoint }`, gdje je `as4_endpoint` naš AS4 inbox iz konfiguracije.

3. Inače → 404.

5.3 B. Izlazni račun

Ovo je glavni tok sustava (kut 2 — posrednik pošiljatelj). Dobavljačev ERP kreira račun, posrednik ga gura kroz životni ciklus, generira i potpisuje UBL, fiskalizira ga prema CIS-u i dostavlja preko AS4 primateljevom posredniku.

Životni ciklus izlaznog računa



Matrica dopuštenih prijelaza (`InvoiceStatus::allowedTransitions`):

Iz \ U	draft	queued	sent	received	delivered	rejected
draft	—	✓				
queued		—	✓			
sent			—		✓	✓
received				—	✓	✓
delivered					terminalno	
rejected						terminalno

received je ulazna grana (cjelina C). **delivered** i **rejected** su terminalni — iz njih nema prijelaza. Svaki nedopušteni prijelaz baca `InvalidInvoiceTransitionException`.

Side-effecti su vezani uz ciljno stanje i smjer (`TransitionInvoiceStatus`):

Prijelaz	Smjer	Side-effect
→ queued	outbound	renderiraj + potpiši + validiraj UBL, pohrani ga (<code>StoreInvoiceUbl</code>)
→ sent	outbound	AS4 dostava (<code>SubmitAs4Delivery</code>) i fiskalizacija (<code>SubmitFiscalization</code>)
→ delivered	inbound	fiskalizacija sa strane primatelja (cjelina C)

5.3.1 UC-B1 — Kreiranje izlaznog računa

Akteri	ERP dobavljača
Cilj	Unijeti novi račun s stavkama; sustav izračuna iznose i sprema ga kao draft .
Endpoint	POST <code>/invoices</code>
Kod	<code>InvoiceController@store</code> → <code>CreateInvoice</code>

Preduvjeti: dobavljač i kupac postoje kao stranke (**parties**).

Glavni tok:

1. ERP šalje zaglavlje (**supplier_oib**, **buyer_oib**, **invoice_number**, **issue_date**, opc. **due_date**, **direction**, opc. **currency**) i polje **lines** (≥ 1 stavka).
2. **StoreInvoiceRequest** validira:
 - oba OIB-a valjana (**Oib**) i postoje (**exists:parties,oib**); kupac različit od dobavljača,
 - **invoice_number** jedinstven po dobavljaču (kombinacija **supplier** + broj),
 - **due_date** (ako je dan) **after_or_equal issue_date**,
 - svaka stavka: **description**, **quantity** > 0, **unit_price** > 0, **vat_rate** 0–100, **vat_category** (S/Z/E), **kpd_code** točno 6 znakova (obavezan), opc. **unit_code** (3 znaka).
3. **CreateInvoice** u transakciji: razriješi dobavljača/kupca po OIB-u, za svaku stavku izračuna **line_total** = **quantity** × **unit_price** i **line_tax** = **line_total** × **vat_rate** / 100 (svi izračuni preko **bcmath**, fiksna decimalna preciznost — bez float grešaka), te zbroji **net_amount**, **tax_amount**, **total_amount**.
4. Račun se sprema sa statusom **draft** (status nije mass-assignable — **Draft** je rodno stanje), zatim se kreiraju stavke. **unit_code** default je H87 (komad).
5. Vraća se 201 **Created** s **InvoiceResource** (s učitanim **supplier**, **buyer**, **lines**).

Posljedice: novi red u **invoices** (status **draft**) + redovi u **invoice_lines**. Iznosi su izvedeni na poslužitelju — ERP ih ne diktira.

Greške: validacijske → 422; nepostojeći OIB → 422 (preko **exists**).

5.3.2 UC-B2 — Pregled i filtriranje računa

Akteri	ERP / administrator
Endpoint	GET /invoices?status=&direction=&supplier_oib=&buyer_oib=
Kod	InvoiceController@index (ListInvoicesRequest)

Glavni tok: paginirana lista (najnoviji prvi), s opcionalnim filtrima po **status**, **direction**, **supplier_oib**, **buyer_oib**. Učitava **supplier**, **buyer**, **lines**.

5.3.3 UC-B3 — Dohvat računa

Endpoint	GET /invoices/{invoice}
Kod	InvoiceController@show

Vraća pojedini račun s relacijama; nepostojeći → 404.

5.3.4 UC-B4 — Preuzimanje UBL XML-a

Akteri	ERP / administrator
Cilj	Dohvatiti UBL prikaz računa (potpisani konačni, ili draft pretpregled).
Endpoint	GET /invoices/{invoice}/xml
Kod	InvoiceXmlController@show → UblDocumentRenderer

Glavni tok:

1. Ako račun ima pohranjeni UBL (`ubl_xml_path` postavljen, dokument se streama s diska) → vrati ga (`Content-Type: application/xml`). To su točni potpisani bajtovi.
2. Inače, ako je račun `draft + outbound` → renderiraj nepotpisani draft UBL on-the-fly (`renderer->draft()`) kao pretpregled mapiranja.
3. Inače → 404 (npr. `inbound` bez pohranjenog dokumenta).

Byte-fidelity: pohranjeni UBL vraća se nepromijenjen — to su točni potpisani bajtovi. Zašto se dokument ne smije ponovno serijalizirati ni „uljepšavati” objašnjeno je uz potpisivanje (4.6).

5.3.5 UC-B5 — Prijelaz statusa (životni ciklus)

Akteri	ERP / administrator
Cilj	Pomaknuti račun kroz životni ciklus; pritom se okidaaju UBL/dostava/fiskalizacija.
Endpoint	PATCH <code>/invoices/{invoice}/status</code>
Kod	<code>InvoiceStatusController@update</code> → <code>TransitionInvoiceStatus</code>

Glavni tok: ERP šalje ciljni status (`UpdateInvoiceStatusRequest` ga validira protiv enuma), a `TransitionInvoiceStatus` izvodi orkestraciju razrađenu u 4.3 (flowchart): provjeri dopuštenost prijelaza, po potrebi renderira i potpiše UBL prije promjene statusa, promijeni status (`transitionTo` kao jedini siguran put) i okine side-effecte iz tablice gore (→ `sent`, `outbound`: AS4 dostava i fiskalizacija). Na uspjeh vraća 200 OK s ažuriranim `InvoiceResource`.

Greške i decoupling: nedopušten prijelaz → iznimka (409/422); nevaljan UBL (XSD/Schematron) → 422 s izvještajem validacije, status ostaje zatečen. Ako *nakon* prijelaza AS4 dostava ili fiskalizacija bace iznimku, prijelaz se ne poništava (decoupling, 4.3) — greška se zapiše na pripadni red i radnja ponavlja preko UC-B6/UC-B7.

5.3.6 UC-B6 — Fiskalizacija računa (ponovni pokušaj / ručno)

Akteri	ERP / administrator (ili automatski iz UC-B5)
Cilj	Prijaviti račun CIS-u i dobiti potvrdu (identifikator poruke, <code>service_message_id</code>).
Endpoint	POST <code>/invoices/{invoice}/fiscalize</code>
Kod	<code>InvoiceFiscalizationController@store</code> → <code>SubmitFiscalization</code>

Glavni tok:

1. Odredi se reporter OIB (`reporterOib()`): za `outbound` to je dobavljač, za `inbound` kupac.
2. **Idempotentnost:** ako već postoji `accepted` fiskalna poruka za taj reporter → `SubmissionOutcome::AlreadyTerminal` → 409 Conflict („already accepted”); ne šalje se ponovno.
3. `FiscalMessageBuilder` izgradi fiskalni XML, koji se potpiše potpisnim vjerodajnicama te stranke (`InvoiceSigner + PartySigningCredentials->forOib`).
4. Kreira se / ažurira fiskalna poruka u stanje `requested` (`request_xml`, `submitted_at`).
5. Poziva se `FiscalizationService->fiscalize(xml)` (HTTP prema mock CIS-u):
 - **uspjeh** → poruka u `accepted`, spremi se `service_message_id`, `match_status`, `settled_at`; vrati 200 OK s resource-om.
 - **greška** (`FiscalizationServiceException`) → poruka u `error` s `error_code`/`error_message`/`settled_at`; baci se `FiscalizationException`.

Posljedice: red u `fiscal_messages` (stanje `requested`→`accepted/error`). Kod ponovnog poziva postojeća se poruka ponovno koristi (`upsert`), ne stvara se duplikat.

5.3.7 UC-B7 — AS4 dostava primatelju (ponovni pokušaj / ručno)

Akteri	ERP / administrator (ili automatski iz UC-B5)
Cilj	Dostaviti potpisani UBL primateljevom posredniku preko AS4.
Endpoint	POST /invoices/{invoice}/deliver
Kod	InvoiceDeliveryController@store → SubmitAs4Delivery

Glavni tok:

1. **Idempotentnost:** ako već postoji `acknowledged` izlazna AS4 poruka → `AlreadyTerminal` → 409 `Conflict` („already acknowledged”).
2. Odrede se `senderOib` (dobavljač) i `recipientOib` (kupac); dohvati se pohranjeni potpisani UBL (`ubl_xml`). Ako ga nema (npr. fixture bez generiranja) → `Skipped` (nema što dostaviti).
3. `As4DeliveryService->send(ubl, sender, recipient):`
 - razriješi primateljev AS4 endpoint (peer discovery — vidi dolje),
 - izgradi i potpiše AS4 envelope, pošalje ga HTTP-om, primi receipt.
4. **Uspjeh:** kreira se izlazni red u `as4_messages` u stanju `acknowledged` (`message_id`, `ref_to_message_id`, envelope + receipt XML, `sent_at`, `acknowledged_at`).
5. **Greška** (`As4DeliveryException`):
 - ako je envelope već bio izgrađen → zapiše se `as4_messages` red u stanju `error` (s envelopeom i `error_code/error_message`) radi revizije,
 - ako je greška prije gradnje envelopea (npr. nepoznat primatelj) → ništa se ne bilježi; baca se `As4DeliveryFailedException`.

Peer discovery: primateljev endpoint razrješava `AmsMpsPeerEndpointResolver` — prvo upita AMS koji MPS opslužuje traženi OIB, pa taj MPS za AS4 endpoint (usp. UC-A5), uz fallback na statičku `OIB=URL` mapu iz konfiguracije (`ConfigPeerEndpointResolver`). Tako sustav radi i bez živog AMS-a.

5.4 C. Ulazni račun

Druga strana razmjene (kut 3 — posrednik primatelj). Posrednik prima tuđi račun — ili izravno od ERP-a (npr. testiranje, isti posrednik na obje strane), ili preko AS4 od drugog posrednika — validira ga, provjerava potpis, sprema kao `received` i dalje ga obrađuje (dostava kupcu + fiskalizacija sa strane primatelja → sravnjivanje).

Rodno stanje ulaznog računa je `received` (ne `draft`). Dalje ide `received` → `delivered` (ili `rejected`), gdje `delivered` znači da je posrednik predao račun kupčevom ERP-u i da je kupac sad obavezan fiskalizirati.

5.4.1 UC-C1 — Prijem računa izravno iz ERP-a

Akteri	ERP (pošiljatelj UBL-a)
Cilj	Zaprimiti gotov UBL račun, provjeriti ga i upisati kao ulazni.
Endpoint	POST /invoices/inbound (tijelo = sirovi UBL XML)
Kod	InboundInvoiceController@store → UblValidator + UblParser + ReceiveInbound

Glavni tok:

1. Tijelo zahtjeva je sirovi UBL XML.

2. **Validacija sheme:** UblValidator->validate() (XSD + Schematron / HR-CIUS). Neispravan dokument → InvoiceValidationException → 422.
3. UblParser->parse() pretvara XML u ParsedInvoice DTO (stranke, broj, datumi, iznosi, stavke).
4. ReceiveInbound->execute(parsed, rawXml):
 - **Provjera potpisa** (assertSignatureValid): dokument mora nositi provjerljiv potpis — digest se podudara (nema neovlaštene izmjene) i certifikat se lančano veže na povjerljivi CA (SignatureVerifier + TrustStore). Neuspjeh → 422.
 - U transakciji: dobavljač se po potrebi auto-kreira (firstOrCreate po OIB-u iz dokumenta), a kupac mora već biti registriran kod ovog posrednika — inače buyer-not-registered → 422.
 - **Idempotentnost:** ako već postoji ulazni račun (isti dobavljač + kupac + broj + inbound) → vrati postojeći (200), bez dupliciranja.
 - Inače kreira Invoice sa statusom received, pohrani sirovi UBL verbatim (StoreInvoiceUbl) i kreira stavke iz parsiranog dokumenta.
5. Vraća 201 Created (novi) ili 200 OK (već postojao) s InvoiceResource.

Posljedice: ulazni račun (received) + pohranjeni UBL + (po potrebi) novi dobavljač.

5.4.2 UC-C2 — Prijem računa preko AS4 (inter-posrednička razmjena)

Akteri	Drugi posrednik (peer access point)
Cilj	Zaprimiti AS4 poruku s UBL teretom, vratiti AS4 receipt ili error.
Endpoint	POST /as4/inbox (tijelo = AS4/SOAP envelope)
Kod	As4InboxController@store → ReceiveAs4Message

Ovo je AS4 protokolni endpoint — uvijek vraća AS4 envelope (application/soap+xml), bilo receipt (HTTP 200) bilo error (HTTP 400) s EBMS:* kodom.

Glavni tok:

1. Parsiranje i XSD validacija envelopea (as4-envelope.xsd). Loše oblikovan/nevaljan → error EBMS:0009.
2. Ekstrakcija metapodataka iz zaglavlja: messageId, fromOib, toOib.
3. Provjera WS-Security potpisa access pointa na envelopeu (digest + lanac do povjerljivog CA). Neuspjeh → EBMS:0009. (*Unutarnji UBL ima vlastiti XAdES potpis koji se provjerava kasnije, pri ingestiji.*)
4. **Idempotentnost (replay):** ako messageId već postoji → vrati spremljeni odgovor (receipt ili error), isti HTTP status.
5. **Primatelj registriran?** Ako toOib nije naša stranka → error EBMS:0005.
6. Zapiše se ulazna AS4 poruka (as4_messages, stanje received).
7. **Ingestija UBL tereta:**
 - izvuče se ubl:Invoice iz tijela envelopea,
 - UblValidator (HR-CIUS) — neuspjeh → EBMS:0004,
 - UblParser + ReceiveInbound (isti tok kao UC-C1, uključujući provjeru XAdES potpisa i kreiranje ulaznog računa); bilo koja greška → EBMS:0004, AS4 poruka u stanje error.
8. **Uspjeh:** izgradi se AS4 receipt, AS4 poruka prelazi u delivered i veže se uz kreirani invoice_id; vrati se receipt (HTTP 200).

Posljedice: red u as4_messages (received → delivered/error); kod uspjeha i ulazni Invoice (received). Pošiljatelj na drugoj strani na temelju receipta svoju izlaznu AS4 poruku označi acknowledged (usp. UC-B7).

Mapiranje grešaka (ebMS): EBMS:0009 (envelope/potpis), EBMS:0005 (nepoznat primatelj), EBMS:0004 (neispravan UBL teret).

5.4.3 UC-C3 — Predaja kupcu i fiskalizacija sa strane primatelja (sraunjivanje)

Akteri	ERP kupca / administrator
Cilj	Predati zaprimljeni račun kupcu i fiskalizirati ga sa strane primatelja.
Endpoint	PATCH /invoices/{invoice}/status (received → delivered)
Kod	TransitionInvoiceStatus → SubmitFiscalization (reporter = kupac)

Glavni tok:

1. Ulazni račun (received) prelazi u delivered — trenutak kad posrednik predaje račun kupčevom ERP-u i kupac postaje obvezan fiskalizirati.
2. Side-effect (inbound + delivered): fiskalizacija sa strane primatelja (SubmitFiscalization, reporter OIB = kupac). Isti mehanizam kao UC-B6.
3. CIS uparuje kupčevu prijavu s ranijom dobavljačevom (ako postoji) i vraća match_status:
 - matched — dobavljač je već prijavio iste iznose,
 - mismatch — dobavljačeva prijava ima različite iznose,
 - pending — dobavljač još nije prijavio.

Sraunjivanje (uparivanje) i živi match_status: posrednik pri prijavi pamti samo ono što mu je CIS tada rekao. Kad kasnije stigne druga strana, CIS interno prebaci oba zapisa u matched/mismatch. Da bi API uvijek pokazao trenutnu istinu, InvoiceResource pri čitanju ponovno pita CIS (FiscalizationService::lookupMatch()) i pada na pohranjenu vrijednost ako upit ne uspije. Tako npr. dobavljačev izlazni račun pokaže pending → matched nakon što kupac fiskalizira, bez ikakvih background jobova ili webhookova.

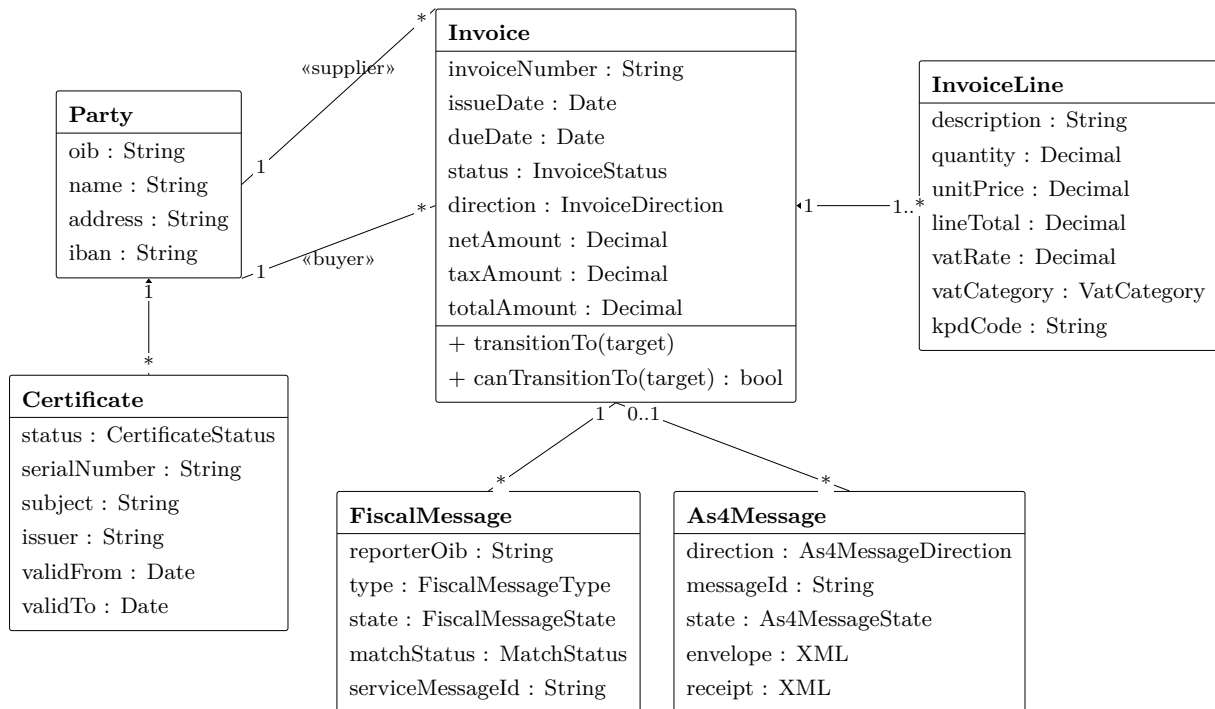
Puni round-trip (jedna instanca, oba posrednika): POST /invoices (draft) → → queued (UBL) → → sent (dobavljačeva fiskalizacija = pending + AS4 dostava) → POST /invoices/inbound ili POST /as4/inbox (ulazni received) → → delivered (kupčeva fiskalizacija) → oba računa pokazuju matched.

6 Model podataka

Baza je relacijska, upravljana Eloquent migracijama. Šest domenskih tablica (uz Laravelove sistemske: users, cache, jobs).

6.1 Klasni dijagram domene

Dijagram prikazuje domenske klase, njihove glavne attribute i međusobne odnose na konceptualnoj razini — bez baznih tipova, indeksa i drugih implementacijskih detalja koji se lako mijenjaju (oni su u tablicama niže). Tipovi stanja (status, state, direction, vatCategory, ...) su enumi, detaljno razrađeni u poglavlju 4.4.



Odnosi i zašto baš takvi:

- **Kompozicija** (◇) **Invoice—InvoiceLine** i **Party—Certificate**: stavke i certifikati ne postoje samostalno bez svog vlasnika — vezani su uz njegov životni ciklus i brišu se kaskadno. Otud puni dijamant na strani cjeline.
- **Asocijacija** **Party—Invoice**, u dvije uloge (*supplier*, *buyer*): stranke žive neovisno o pojedinom računu, koji ih samo referencira. Ista **Party** može biti dobavljač na jednom, a kupac na drugom računu.
- **Invoice—FiscalMessage** i **Invoice—As4Message** su asocijacije vezane uz tijek računa. **As4Message** na strani računa ima višestrukost 0..1 jer poruka može nastati i *prije* vezi- vanja uz račun (greška u dostavi prije razrješenja).
- **Bez nasljeđivanja**: varijante stanja i tipova modelirane su enumima (poglavlje 4.4), a ne potklasama — stanje je podatak, ne tip, pa nema hijerarhije klasa.

6.2 Tablice i ključne invarijante

Šest domenskih tablica prati klase iz dijagrama (uz Laravelove sistemske **users**, **cache**, **jobs**). Atributi su vidljivi na dijagramu; ovdje su istaknute samo invarijante na razini baze koje se iz njega ne vide:

Tablica	Ključno ograničenje / napomena
parties	oib je unique; country_code default HR.
invoices	Iznosi (net/tax/total) izračunati na serveru (decimal, bcmath), currency default EUR. ubl_xml_path drži putanju do UBL-a na disku — sadržaj ubl_xml je read-only accessor koji streama dokument verbatim (byte-fidelity; vidi UC-B4 i 4.6).
invoice_lines	FK na invoices (cascade); kpd_code 6 znamenki, unit_code default H87.
fiscal_messages	unique (invoice_id, reporter_oib, message_type) — svaka strana ima točno jednu FIS poruku po računu (temelj idempotentnog upserta, UC-B6); request/response_xml su revizijski trag.
as4_messages	message_id unique → replay-idempotentnost prijema (UC-C2); invoice_id nullable (poruka može nastati prije vezivanja uz račun).
certificates	FK na parties (cascade); index (party_id, status) pogoni „single-active” invarijantu. certificate_pem je javni; private_key_path pokazuje na ključ na PKI disku i nikad se ne serijalizira.

7 Korisničke upute

eRakun namjerno nema grafičko sučelje — sustav je REST API koji opslužuje strojne klijente (ERP-ove i druge posrednike). Za potrebe demonstracije i ovih uputa kao sučelje (prototip) koristimo Bruno — open-source, offline API klijent. Bruno kolekcija živi u repozitoriju (bruno/eRakun/) kao plain-text .bru datoteke, pa je verzionirana i čitljiva u diffu, a istovremeno služi kao ručno upravljivo sučelje nad svakim endpointom. Upute u nastavku prate kanonski tijek izrade i razmjene jednog eRačuna, korak po korak.

O snimkama zaslona. Svaki korak ima mjesto za snimku Bruno sučelja (zahtjev + odgovor). U izvoru su to \screenshot{...} rezervirana mjesta; kad je snimka spremna, zamijeni poziv s npr.

```
\includegraphics[width=.86\textwidth]{slike/01-create-party.png}.
```

7.1 Sučelje: Bruno API kolekcija

Otvaranje kolekcije:

1. Instaliraj Bruno (`brew install -cask bruno` ili s `usebruno.com`).
2. **Open Collection** → odaberi mapu `bruno/eRakun`.
3. Gore desno odaberi okolinu **Local**.

Okolina **Local** drži zajedničke varijable koje zahtjevi dijele:

Varijabla	Zadana vrijednost / uloga
base_url	<code>http://localhost:8000/api</code>
supplier_oib	12345678903 — OIB dobavljača
buyer_oib	98765432106 — OIB kupca
certificate_id	popunjava se automatski nakon <i>Upload Certificate</i>
invoice_id	popunjava se automatski nakon <i>Create Invoice</i>

Stranke se u rutama adresiraju po OIB-u (`/parties/{oib}`), a računi po internom `id` (`/invoices/{id}`) koji se sprema u `invoice_id` skriptom nakon kreiranja.

[screenshot: Bruno glavni prozor — kolekcija eRakun, odabrana okolina Local]

zamijeniti s \includegraphics stvarne snimke

7.2 Priprema okoline

Prije prvog zahtjeva pokreni poslužitelj i pripremi bazu te testni PKI:

```
php artisan migrate          # ili migrate:fresh za čistu bazu
php artisan pki:generate --parties # izda potpisne certifikate subjektima
php artisan serve            # http://localhost:8000
```

`pki:generate --parties` je najlakši način da dobavljač dobije aktivan potpisni certifikat (alternativa je ručni upload `.p12` u koraku 2). Bez certifikata potpisani UBL i fiskalizacija ne rade. Fiskalizacija i AS4 dostava dodatno traže companion servise (`FISCALIZATION_SERVICE_URL`, `AS4_DEFAULT_PEER_URL`); koraci 1–4 rade i bez njih.

Cijela priprema sažeta je u `composer setup` (install + `.env` + ključ + migracije + kompilacija shema), a razvojni poslužitelj se diže s `composer dev`. Provjere kvalitete (uskladiti prije predaje): `composer check` (Pint + Rector + PHPStan + testovi), `composer test`, `composer lint`.

7.3 Tijek rada (pregled)

Preporučeni redoslijed mapa u kolekciji prati životni ciklus računa:

1. **01 Parties** — registriraj dobavljača i kupca.
2. **02 Certificates** — provjeri/učitaj potpisni certifikat dobavljača.
3. **03 Invoices** — kreiraj račun, prebaci u `queued` (potpisani UBL), dohvati XML.
4. **04 Fiscalization & Delivery** — fiskaliziraj prema CIS-u i dostavi preko AS4.
5. **05 Inbound & Network** — zaprimanje dolaznog računa i mrežne usluge (MPS).

7.4 Korak 1 — Registracija stranaka

U mapi **01 Parties** pošalji *Create Party - Supplier*, zatim *Create Party - Buyer*. Zahtjev je `POST /api/parties` s podacima subjekta:

```
{ "oib": "12345678903", "name": "TVRTKA A d.o.o.", "is_vat_registered": true,
  "address_line": "Ulica 1", "city": "ZAGREB", "postcode": "10000",
  "country_code": "HR", "iban": "HR1723600001101234565" }
```

Odgovor 201 Created vraća `PartyResource`. `oib` mora proći ISO 7064 (MOD 11,10) kontrolu i biti jedinstven, inače slijedi 422. Provjeri stranku zahtjevom *Get Party* (`GET /api/parties/{{supplier_oib}}`) — adresira se po OIB-u, ne po id.

[screenshot: Bruno — Create Party (Supplier): JSON zahtjev i odgovor 201]

zamijeniti s \includegraphics stvarne snimke

7.5 Korak 2 — Potpisni certifikat dobavljača

Dobavljač potpisuje svoje račune, pa mora imati aktivan certifikat. Ako si pokrenuo `pki:generate -parties`, certifikat već postoji — potvrdi to s *List Certificates* (GET `/api/parties/{supplier_oib}/certificates`, najnoviji prvi).

Ručno učitavanje (*Upload Certificate*, POST `.../certificates`) ide na dva načina: PKCS#12 datoteka (polje `certificate`, `multipart/form-data`, uz `password` ako je zaštićen) ili PEM string u JSON polju `certificate_pem`. Na uspjeh (201) kolekcija sprema `data.id` u `certificate_id`. Opoziv ide preko *Revoke Certificate* (DELETE, status → `revoked`, 204).

Privatni ključ nikad nije dio odgovora — `CertificateResource` izlaže samo javni `certificate_pem` i metapodatke (subject, issuer, rok valjanosti, fingerprint).

[screenshot: Bruno — List Certificates: aktivan certifikat dobavljača]

zamijeniti s \includegraphics stvarne snimke

7.6 Korak 3 — Kreiranje izlaznog računa

U mapi **03 Invoices** pošalji *Create Invoice* (POST `/api/invoices`). Tijelo je strukturirani JSON; posrednik sam izračunava neto, PDV i ukupno iz stavki:

```
{ "supplier_oib": "12345678903", "buyer_oib": "98765432106",
  "invoice_number": "RN-2026-00042", "issue_date": "2026-04-15",
  "due_date": "2026-05-15", "direction": "outbound", "currency": "EUR",
  "lines": [ { "description": "Konzultantske usluge", "quantity": 1,
    "unit_price": 100.00, "vat_rate": 25.00, "vat_category": "S",
    "unit_code": "H87", "kpd_code": "622020" } ] }
```

Račun nastaje u statusu `draft`; odgovor 201 sprema `data.id` u `invoice_id`. Ključna ograničenja: dobavljač i kupac moraju postojati i biti različiti, `invoice_number` je jedinstven po dobavljaču, a `kpd_code` mora biti 6 znamenki iz CPA liste (npr. 622020, 960212) — zahtjev provjerava samo duljinu, ali nevaljan kod puca tek u koraku 4 (Schematron HR-BR-25).

[screenshot: Bruno — Create Invoice: zahtjev i odgovor s izračunatim iznosima]

zamijeniti s \includegraphics stvarne snimke

7.7 Korak 4 — Generiranje i potpisivanje UBL-a

Pošalji *Update Invoice Status* (PATCH /api/invoices/{{invoice_id}}/status) s tijelom { "status": "queued" }. Prijelaz draft → queued (samo za outbound) je onaj koji generira UBL 2.1 / HR-CIUS i potpisuje ga (XAdES-B) te ga sprema na račun. Tek nakon toga *Get Invoice XML* (GET .../xml, Accept: application/xml) vraća potpisani dokument, a dostava ima što slati.

Ako stavka nosi KPD kod izvan CPA liste, ovdje slijedi 422 (EN 16931 / HR-BR-25), a status ostaje draft. Dopušteni prijelazi: draft → queued → sent → delivered/rejected (matrica u poglavlju *Use case-ovi*); nedopušten prijelaz vraća grešku.

[screenshot: Bruno — Get Invoice XML: potpisani UBL (ds:Signature blok)]
zamijeniti s \includegraphics stvarne snimke

7.8 Korak 5 — Fiskalizacija i AS4 dostava

U mapi **04 Fiscalization & Delivery**:

- *Fiscalize Invoice* (POST .../fiscalize) — šalje fiskalni zapis u CIS (mock Porezne) i bilježi ishod na računu. 200 = prihvaćeno; 409 = već u završnom stanju.
- *Deliver Invoice (AS4)* (POST .../deliver) — slaže AS4/SOAP poruku s potpisanim UBL-om i bilježi potvrdu. Traži potpisani UBL (korak 4) i živu peer pristupnu točku; bez peera 422 „AS4 peer rejected the delivery.“.

Rezultat čitaš kroz *Get Invoice* (GET /api/invoices/{{invoice_id}}): blok **fiscalization** (npr. state: accepted, match_status: matched) i blok **delivery** (npr. state: acknowledged). Oba su null dok pripadna poruka ne postoji.

[screenshot: Bruno — Get Invoice nakon fiskalizacije i dostave (blokovi fiscalization i delivery)]

zamijeniti s \includegraphics stvarne snimke

7.9 Korak 6 — Zaprimanje i mrežne usluge

Mapa **05 Inbound & Network** pokriva primateljsku stranu:

- *Receive Inbound UBL* (POST /api/invoices/inbound, tijelo je sirovi UBL XML) — posrednik validira UBL/HR-CIUS i potpis, parsira i pohranjuje kao **inbound** račun (received). Priloženi uzorak iz repoa nije potpisan pa kako jest vraća 422 „The document carries no ds:Signature.“; valjani ulaz je potpisani XML iz koraka 4 (idealno s druge instance).
- *MPS Participant Lookup* (GET /api/mps/participants/{{supplier_oib}}) — za zadani OIB vraća **as4_endpoint** (analogno PEPPOL SML/SMP). 404 ako OIB nije opslužen.

- *AS4 Inbox* (POST /api/as4/inbox) — server-to-server prijemna točka; u normalnom radu je ne pozivaš ručno, nego je zove suprotni posrednik.

[screenshot: Bruno — MPS Participant Lookup: oib i as4_endpoint]

zamijeniti s \includegraphics stvarne snimke

7.10 Čitanje statusa i česte greške

Simptom	Uzrok / rješenje
422 pri kreiranju računa	Nepostojeći ili jednaki OIB-ovi, dupli <code>invoice_number</code> , kriv format polja.
422 pri → <code>queued</code>	KPD kod nije u CPA listi (HR-BR-25) ili UBL ne prolazi Schematron; status ostaje <code>draft</code> .
<i>Deliver</i> vraća 200 bez učinka	Račun nema potpisani UBL — prvo ga prebaci u <code>queued</code> (korak 4).
<i>Deliver</i> / <i>Fiscalize</i> transportna greška	Companion servis (peer / CIS mock) nije pokrenut ili je kriv URL u <code>.env</code> .
<i>Receive Inbound</i> 422 (nema potpisa)	Ulazni UBL mora biti potpisani dokument iz <i>Get Invoice XML</i> .
409 pri <i>Fiscalize</i> / <i>Deliver</i>	Radnja je već u završnom stanju (idempotentno; nema dvostruke prijave).

Puni round-trip između dvije instance. Pokreni drugu instancu eRakuna kao primatelja (zaseban `.env`/port, vidi `.env.peer`) i usmjeri AS4/peer konfiguraciju jedne na drugu; račun tada prođe Supplier ERP → Sender AP → Receiver AP → Buyer ERP. Automatizirano u `tests/Integration/TwoInstanceRoundTripTest.php`.

8 Zaključak i ograničenja

8.1 Što projekt postiže

eRakun implementira funkcionalnu jezgru informacijskog posrednika za Fiskalizaciju 2.0, pokrivajući cijeli 4-corner tijekom razmjene eRačuna — od ERP integracije, preko UBL 2.1 / HR-CIUS, potpisivanja i verifikacije na dva sloja i fiskalizacije sa savnjivanjem, do AS4 razmjene i dinamičkog otkrivanja primatelja. Opseg je razrađen u poglavlju 1.3 i kroz use case-ove, pa se ovdje ne ponavlja.

Vrijednost rješenja je u dosljednoj slojevitoj arhitekturi (4.2, 4.5): domena odvojena od HTTP-a i transporta, a vanjski servisi zamjenjivi mock ↔ produkcija bez diranja domene. Tome se pridružuju čvrste invarijante: dopušteni prijelazi stanja, single-active certifikat i idempotentne radnje.

8.2 Svjesna ograničenja

Sve navedeno su namjerne granice opsega studentskog projekta — prikladne ovdje, ali označene da se ne prenose u nešto stvarno:

Područje	Ograničenje
Porezna / CIS PKI	Oponašano companion aplikacijom (erakun-porezna), nije pravi CIS. Samo-generirani testni korijeni umjesto pravih FINA/OpenPEPPOL certifikata.
Opoziv certifikata	Nema OCSP/CRL provjere; opoziv je samo interni status revoked .
Vremenski žig eReporting	XAdES nosi SigningTime , ali nema kvalificiranog TSA (XAdES-T i više). Samo osnovni fis tip; plaćanja/odbijanja (fis_payment/fis_reject/fis_ir) predviđeni ali neimplementirani.
Sigurnost API-ja	Nema autentikacije/autorizacije ERP klijenata, rate-limitinga ni multi-tenancyja.
Otpornost	Dostava/fiskalizacija se ponavljaju ručno (retry endpointi); nema automatskih redova/poslova.

9 Literatura i izvori

Standardi i specifikacije

- **EN 16931-1:2017** — *Electronic invoicing — Part 1: Semantic data model of the core elements of an electronic invoice*. CEN.
- **OASIS UBL 2.1** — *Universal Business Language Version 2.1*. OASIS Standard, 2013. <https://docs.oasis-open.org/ubl/UBL-2.1.html>
- **HR-CIUS / HR-FISK 2.0** — Hrvatska prilagodba (*Core Invoice Usage Specification*) norme EN 16931; Porezna uprava, Fiskalizacija 2.0. <https://porezna.gov.hr/fiskalizacija>
- **OASIS AS4 / ebMS 3.0** — *AS4 Profile of ebMS 3.0 Version 1.0*. OASIS Standard, 2013. <https://docs.oasis-open.org/ebxml-msg/ebms/v3.0/profiles/AS4-profile/v1.0/os/AS4-profile-v1.0-os.html>
- **OASIS WS-Security** — *Web Services Security: SOAP Message Security 1.1.1*.
- **W3C XML Signature** — *XML Signature Syntax and Processing (XMLDSig)*. W3C Recommendation. <https://www.w3.org/TR/xmlsig-core/>
- **ETSI EN 319 132-1** — *XAdES digital signatures; Part 1: Building blocks and XAdES baseline signatures*. ETSI. https://www.etsi.org/deliver/etsi_en/319100_319199/31913201/01.03.01_60/en_31913201v010301p.pdf
- **KPD 2025** — Nacionalna klasifikacija proizvoda po djelatnostima; Državni zavod za statistiku.

Alati i biblioteke

- **Laravel** — PHP web framework. <https://laravel.com/docs>
- **robrichards/xmlseclibs** — PHP biblioteka za XML-DSig / XAdES. <https://github.com/robrichards/xmlseclibs>
- **OpenSSL** — kriptografske primitive (RSA, SHA-256, X.509, PKCS#12). <https://www.openssl.org/>